Lex and Yacc Parser: Assignment, if, do-while

lexer.l

```
%{
#include "y.tab.h"
%}

%%

"if"                        { return IF; }
"else"                      { return ELSE; }
"do"                        { return DO; }
"while"                     { return WHILE; }
[a-zA-Z_][a-zA-Z0-9_]*      { return ID; }
[0-9]+                      { return NUM; }
"<="|">="|"=="|"!="|"<"|">" { return RELOP; }
[ \t\n]+                    ;
.                           { return yytext[0]; }

%%

int yywrap() { return 1; }
```

parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
%}

%token IF ELSE DO WHILE ID NUM RELOP

%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE
%left '+' '-'
%left '*' '/'

%%

program   : stmt_list           { printf("Valid syntax\n"); }
          ;

stmt_list : stmt_list stmt
          | stmt
          ;

stmt      : assign
          | ifstmt
          | dowhile
          | '{' stmt_list '}'
          ;

assign    : ID '=' expr ';'
          ;

ifstmt    : IF '(' cond ')' stmt %prec LOWER_THAN_ELSE
          | IF '(' cond ')' stmt ELSE stmt
          ;
dowhile   : DO stmt WHILE '(' cond ')' ';'
```

```
          ;

cond      : expr RELOP expr
          | expr
          ;

expr      : expr '+' expr
          | expr '-' expr
          | expr '*' expr
          | expr '/' expr
          | '(' expr ')'
          | ID
          | NUM
          ;

%%

void yyerror(const char *s) {
    printf("Invalid syntax\n");
}

int main() {
    printf("Enter statements (Ctrl+D to end):\n");
    yyparse();
    return 0;
}
```

How to run

```
lex lexer.l
yacc -d parser.y
gcc lex.yy.c y.tab.c -o parser
./parser
```

Sample inputs

```
a = b + 5;

if (a < b) x = 1; else x = 2;

do { i = i + 1; } while (i < 10);
```

Output

```
Valid input   : Valid syntax
Invalid input (e.g. a = ;) : Invalid syntax
```